

Literature Review: Jackson Investments Financial Advisor Bot

Introduction

In this review I examine five peer-reviewed papers that are directly relevant to the two-stage architecture at the core of this project. The system I am building uses a Long Short-Term Memory (LSTM) model to generate directional signals from stock price data, and a reinforcement learning (RL) agent to make portfolio-level buy, sell, or hold decisions based on those signals. The papers cover three areas: foundational RL portfolio management, LSTM-based stock price prediction, and technical indicator feature engineering. Together they provide the evidence and design rationale behind the key choices I have made in the proposal.

1. A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem

Jiang, Z., Xu, D., & Liang, J. (2017). [arXiv:1706.10059](#). (439 citations)

This paper establishes the foundational framework for treating portfolio management as a deep reinforcement learning problem. The authors argue that prediction-based approaches, which first forecast prices and then apply hand-coded rules to trade, are inherently limited because the prediction and decision layers are decoupled. Their model-free RL approach removes this dependency entirely and learns a policy that maps market state directly to portfolio weights. The authors state that "the framework is not restricted to any particular markets" (p. 3), so while the experiments use cryptocurrency data, the design is intended to be general.

The core of the framework is the Ensemble of Identical Independent Evaluators (EIIE) topology. As the paper describes, "an IIE is a neural network whose job is to inspect the history of an asset and evaluate its potential growth for the immediate future" (p. 2). Each IIE takes the price history of a single asset as input and outputs a growth potential score. All IIEs share parameters, so the network learns a universal asset-evaluation function rather than asset-specific rules. This makes the architecture scalable to any number of assets without retraining from scratch. The scores are passed through a softmax layer to produce normalised portfolio weights. Three IIE types are tested: a CNN, a basic RNN, and an LSTM. A Portfolio-Vector Memory (PVM) stores the output weights across periods so the agent can account for transaction costs during training.

The 11 highest-volume non-cash cryptocurrencies are chosen for the portfolio in the experiments. The reason is that higher trading volume implies better market liquidity (p. 8). I plan to apply the same logic when selecting equities from the TSX, NYSE, and NASDAQ. Choosing stocks with sufficient daily trading volume will make my back-test assumptions more realistic and reduce the chance that the system produces recommendations that would be difficult to execute.

All three EIIE variants outperform traditional portfolio strategies in back-tests on the Poloniex exchange. They achieve at least a 4-fold return over 50 days at a 0.25% commission rate. The LSTM variant actually underperformed the CNN and basic RNN. The authors suggest this may be because vanilla RNN retains more historical context without selective forgetting, which lets it exploit repeating price patterns more effectively. This is worth keeping in mind for my project, when LSTM is used inside a larger RL system rather than as a standalone predictor, a simpler architecture may perform better.

The main practical difference between this paper and my project is the trading frequency. This framework uses a 30-minute window while I am using daily OHLCV data from yfinance. The authors do not test whether the approach holds up at daily granularity, so it is an open question whether the EIIE's ability to exploit short-term patterns carries over to longer time time-frames. Zou et al. (2024) provide some evidence that it does by applying a closely related architecture to daily equity data across four international markets with strong results.

2. Study on the Prediction of Stock Price Based on the Associated Network Model of LSTM

Ding, G., & Qin, L. (2020). *International Journal of Machine Learning and Cybernetics*, 11(6), 1307–1317. (256 citations)

Ding and Qin point out that LSTM stock prediction models typically predict a single price value and treat open, high, low, and close as independent targets, even though these values are causally related within the same trading day. Their associated neural network model addresses this by using three parallel LSTM-based deep recurrent neural network (DRNN) branches whose outputs feed into each other. The first branch predicts the opening price. The second takes the opening price prediction as additional input and predicts the lowest price. The third uses both opening and lowest price predictions to predict the highest price. The total loss is the average of the three branch losses and the entire model is trained end-to-end.

The model was tested on the Shanghai composite index, PetroChina, and ZTE. The model achieves above 95% accuracy and outperforms standalone LSTM and LSTM-DRNN baselines. The architecture encodes the logical structure of intraday price formation: the high cannot be lower than the low, and both depend on where the stock opened.

This is relevant to how I structure the input features for Stage 1 of my project. The LSTM signal generator will use OHLCV data, and treating these values as related rather than independent may improve the quality of the directional signal it produces. One limitation is that the experiments use Chinese equity indices, which have a regulatory 10% daily price movement cap. This cap simplifies the conditional structure of the model, particularly for the branch predicting highest price. It is unclear whether the same approach would work as well on less-constrained markets like the TSX, NYSE, or NASDAQ.

3. A Novel Deep Reinforcement Learning Based Automated Stock Trading System Using Cascaded LSTM Networks

Zou, J., Lou, J., Wang, B., & Liu, S. (2024). Expert Systems with Applications, 242, 122801. (60 citations)

This is the most architecturally similar implementation to the two-stage design I am proposing. Zou et al. argue that standard DRL methods were originally built for games and cannot capture temporal dependencies in financial time-series data. They process raw state vectors using MLP or convolutional layers that have no memory. Their solution is the CLSTM-PPO model. It uses a cascaded structure where an LSTM (LSTMpre) processes a rolling window of 30 market state vectors first and then compresses them into a 128-dimensional feature encoding. This encoding is then passed to a PPO-based RL agent whose actor (LSTMact) and critic (LSTMcri) are also LSTM networks.

The state vector is 181-dimensional and includes available balance, stock prices, shareholdings, and four technical indicators (MACD, RSI, CCI, and ADX) for 30 stocks. The agent can buy, sell, or hold each stock, and the reward is the change in total portfolio value per trading period.

The model is tested across four markets (the Dow Jones 30, SSE 50, SENSEX, and FTSE 100) and consistently outperforms six baselines including a published ensemble strategy using PPO, A2C, and DDPG. In the Chinese market the CLSTM-PPO achieves a cumulative return of 222.91% against 120.87% for the next-best method. The paper also finds that a 30-day lookback window (TW=30) for LSTMpre produces the best results. I will use this as the starting point when choosing a rolling window length for my Stage 1 LSTM.

The main difference from my project is that CLSTM-PPO uses LSTM purely as a temporal encoder inside the RL system rather than as a separate upstream prediction model. I am separating these concerns deliberately: my LSTM is trained to output a directional signal and the RL agent is trained separately to act on that signal in the context of the portfolio. Zou et al. show that the combined LSTM-PPO approach works on equity markets, which validates my choice of PPO as the algorithm for Stage 2.

4. Key Technical Indicators for Stock Market Prediction

Mostafavi, S. M., & Hooman, A. R. (2025). Machine Learning with Applications, 20, 100631. (21 citations)

This paper fills a gap that I noticed when reading the wider literature: most studies pick a small set of technical indicators without explaining why those specific ones were chosen. Mostafavi and Hooman test 88 indicators as features for predicting the S&P 500 daily closing price across 73 years of data (1950–2023). They use four models XGBoost, Random Forest, SVR, and LSTM, with PCA-based dimensionality reduction applied before training.

The LSTM model outperforms the other three by a wide margin. Without a one-day price lag it achieves an MAE of 0.014, compared to 1.39 for Random Forest, 4.89 for XGBoost, and 7.76 for SVR. This gives me a lot of confidence in choosing LSTM over tree-based or kernel-based alternatives for Stage 1.

The study analyzes the importance of different features using SHAP values to identify the most predictive indicators for momentum, trend, volatility, and volume. These are the most important features for each:

- **Momentum:** HLC3 (average of high, low, and close), KAMA, SMA variants
- **Trend:** TEMA, FWMA, EMA, HMA, Ichimoku Cloud, HWMA
- **Volatility:** Bollinger Bands, ATR, Price Distance
- **Volume:** OBV, PVT, MFI

HLC3, TEMA, FWMA, OBV, and Bollinger Bands appear as important features across all four model types, which suggests they carry real market impact. Comparing this to the features I am planning for Stage 1 (RSI, MACD, Bollinger Bands, ATR, and volume ratios) there is good overlap on volatility (Bollinger Bands, ATR) and volume (volume ratios are similar to OBV and PVT). RSI and MACD rank lower in this paper, but they appear as important state features in Zou et al. (2024), so their value may depend on how they are used rather than being generally weak.

5. Prediction of Stock Price Based on LSTM Neural Network

Wei, D. (2019). 2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM), 544–547. (multiple citations)

Wei's paper gives me a baseline for LSTM predictions applied to equity indexes with an attention layer added on top. The attention mechanism lets the network learn which time steps are most relevant when forming a prediction. Six input features are used: date, closing price, opening price, maximum price, minimum price, and daily volume. The model is trained on the Shanghai composite, Shenzhen composite, and CSI 300 indices. It uses Mini-Batch Gradient Descent with a batch size of 64 and a learning rate of 0.001.

The most useful finding for my project is that the model shows a time-lag effect when the input window is short. The predicted peaks and troughs trail the real ones by several periods. As the input window grows from one year to two years, this lag reduces significantly. This is directly relevant to Stage 1 of my project. If I use a rolling window that is too short, the LSTM will generate directional signals that are already outdated by the time they reach Stage 2, which will produce stale buy and sell recommendations and hurt overall performance.

Compared to Ding & Qin (2020), Wei's model is simpler but makes a more useful contribution by specifically testing window size and documenting how it affects lag. Ding & Qin fix their step size at 20 without exploring this in depth. Both papers work only on Chinese equity indices and neither tests

generalisation across different market regimes. This is a gap I need to address through the back-testing module, which will test the system on data that was not used during training.

Conclusions

Reading across all five papers, a few consistent findings stand out. First, LSTM outperforms simpler architectures on financial time-series tasks in every paper that makes the comparison. Wei (2019), Ding & Qin (2020), and Mostafavi & Hooman (2025) all reach the same conclusion. Second, the separation between prediction and decision-making that I have designed into the project is supported by Jiang et al. (2017) and Zou et al. (2024). Both papers show that RL agents working from temporal market representations outperform agents working from raw features or hand-coded rules. Third, input window length and feature selection both have measurable effects on model quality. The 30-day window identified by Zou et al. (2024) and the lag findings in Wei (2019) both point toward using at least 20 to 30 trading days as the lookback period for Stage 1.

One key difference between the papers and my approach is how tightly the prediction and decision layers are connected. Jiang et al. (2017) and Zou et al. (2024) train both layers together as one system. I am training them separately, with the LSTM producing a signal that the RL agent then acts on. This separation makes the system easier to interpret since I can examine the LSTM signal on its own. The trade-off is that the LSTM is not trained to produce signals that specifically help the RL agent maximise returns, so the combined performance may be lower than a fully integrated approach. This is a limitation I have recognized and something I would look to explore further if I extend the project.

References

- Jiang, Z., Xu, D., & Liang, J. (2017). A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem. arXiv:1706.10059.
- Ding, G., & Qin, L. (2020). Study on the prediction of stock price based on the associated network model of LSTM. *International Journal of Machine Learning and Cybernetics*, 11(6), 1307–1317.
- Zou, J., Lou, J., Wang, B., & Liu, S. (2024). A novel Deep Reinforcement Learning based automated stock trading system using cascaded LSTM networks. *Expert Systems with Applications*, 242, 122801.
- Mostafavi, S. M., & Hooman, A. R. (2025). Key technical indicators for stock market prediction. *Machine Learning with Applications*, 20, 100631.
- Wei, D. (2019). Prediction of Stock Price Based on LSTM Neural Network. *2019 International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM)*, 544–547.